Navigating Paris' Metro

Graph Theory Project Report by LESTERLIN Raphaël, DONNAT Arthur and DELANNOY Arthur

Context

The Floyd-Warshall algorithm, applied on a graph, allows after some lengthy calculations to obtain two matrices which give all possible minimum-cost paths between one vertice and another one in the whole graph. Graphs are often used to modelize in a quick and visually understandable way problems of transportation, be it through research operations, or in the case we will study axis of transit throughout a given geographical unit.

Paris has the 2nd most dense metropolitan network in the whole world [1], it is the perfect transportation system to apply this algorithm on. We will thus try to find the minimum cost path across the network to get from point A to point B.

Data

To keep a reasonable scope for this project, we immediately put some constraints on our attempt due to the massive number of stations and lines across the network. We decided to restrain the vertices to the set of stations providing a connection to other lines of metro, in the fourteen historical lines of the Parisian metro. Our cost will be modelized by the number of stations in-between a stop and another one, as we do not have the time to empirically check the average time for an ideal rolling stock to go towards the destination. Using the RATP website [2], we put in an excel sheet the list of connected stations for each line, identifying with a unique ID each station. Then, we used this data to create the graph as a list of arcs from one transit nexus to the next. The file containing it will be given as an appendix (métro correspondances and metro correspondances vertices), since it would be too long to fit in this document in an appropriate manner.

Results

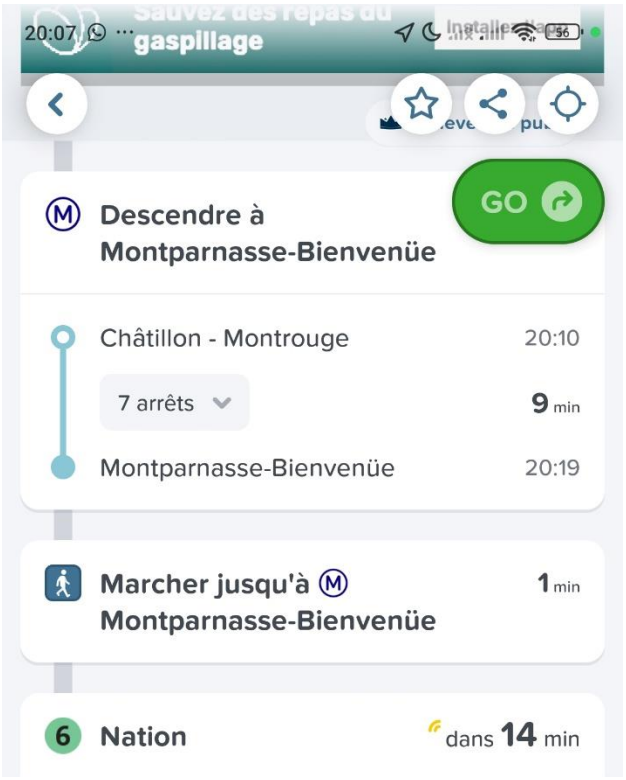
Running our algorithm on the graph, we an extremely long execution trace. Due to the number of vertices, the simple act of calculating the minimum path matrices pushed the weight of the txt file to 15 Gigabytes. As such, it is not possible to display in a single word document all the steps. However, it is left to the reader via this hyperlink to try to have a look himself to the details.

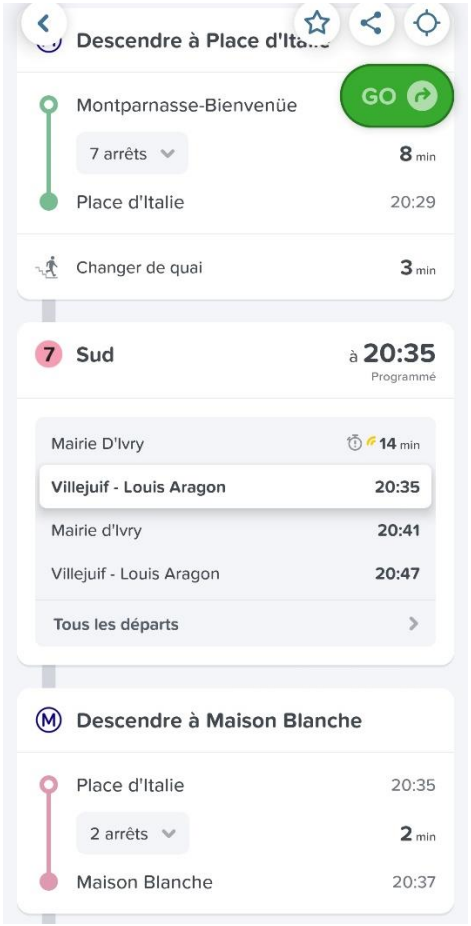
To test our program's accuracy, we determine the minimum cost path (the "shortest") and compare to the propositions of apps (Google Maps, City Mapper) to see if they are both coherent.

We tested first with a Chatillon-Montrouge to Maison Blanche trajectory, which yielded the following :

```
The minimum-value path from 120 to 70 has a value of 16 and goes as follows:
    120 (3) -> 119 (4) -> 47 (2) -> 48 (1) -> 49 (4) -> 56 (2) -> 70
```

To substitute with the name of stations, the given path is Chatillon-Montrouge -> Porte de Vanves -> Montparnasse Bienvenue -> Raspail -> Denfert-Rochereau -> Place d'Italie -> Maison Blanche. Now, to compare with a metro only path given by city mapper we obtain :





We can see that globally, the path given by our program follows the same idea as the citymapper trajectory : going up line 13, then taking line 6 to join line 7 at place d'italie.

To continue the analysis, we will try with 4 other combinations, then directly write the proposed trajectories by both.

Start and end	Program's proposition	CityMapper's
Champs Elysées – Pont de Sèvres	Line 1 to 9 (via Franklin D Roosevelt) -> 15 stations in between	Line 13 to 9 (change via Miromesnil) -> 16 stations in between
Odéon - Châtelet	Line 10 to 4 -> 3 stations	Same
Balard – Porte de Choisy	Line 8 to 6 (La motte Picquet) to 7 (Place d'italie) -> 20 stations	Line 8 to 7 (via opera) -> 26 stations
Rosny Bois Perrier – Porte de champerret	Line 11 to 2 (via Belleville) to 3 (Villiers) -> 27 stations	Line 11 to 3 (via Arts & Métiers) -> 28 stations

And for the record, here are the results yielded by the program :

```
The minimum-value path from 4 to 89 has a value of 15 and goes as follows:
        4 (1) -> 3 (1) -> 93 (2) -> 57 (2) -> 92 (3) -> 91 (1) -> 90 (5) -> 89
```

```
The minimum-value path from 46 to 7 has a value of 3 and goes as follows:
        46 (1) -> 45 (2) -> 7
```

```
The minimum-value path from 79 to 72 has a value of 19 and goes as follows:
        79 (5) -> 59 (2) -> 101 (1) -> 47 (2) -> 48 (1) -> 49 (4) -> 56 (2) -> 70 (1) -> 71 (1) -> 72
```

```
The minimum-value path from 104 to 27 has a value of 27 and goes as follows:
        104 (7) -> 38 (2) -> 77 (3) -> 24 (2) -> 23 (1) -> 22 (1) -> 21 (1) -> 20 (1) -> 19 (1) -> 18 (2) -> 17 (2) -> 16 (3) -> 28 (1) -> 27
```

Conclusion

Over the few examples we considered, we can see definitively that the shortest path in terms of time, easily estimated by apps such as CityMapper, isn't that much linked to the number of stations a passenger has to go through. If we aim to shorten the travel by minimizing the number of stations gone through, it can even lead to making more line changes than necessary (Balard – Porte de Choisy reveals that our program makes us go through 3 lines, while CityMappers makes us use only two). And the difference, as shown by the last tested case study, can happen only by a difference of one station. Despite the reality that our program can't be used like a GPS system in production, it succeeds its goal : finding the minimal path in terms of stations through the parisan metro network.

Bibliography

[1] Île-de-France Mobilités : orchestrer la plus grande révolution des mobilités en Europe | Île-de-France Mobilités

[2] Plan des lignes du métro, RER, bus et tramway | RATP